



BSc (Hons) in **Information Technology**  
**Specialized in AI**  
**IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 02

**Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

### Part 1: Practical Implementation — Coding the Architectures

#### Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates ( `agent_pos` ). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

#### Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food\_here THEN suck; IF wall\_ahead THEN turn\_left; ELSE move\_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

### Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.  
Example Logic: IF wall\_ahead AND left\_is\_visited THEN turn\_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

## Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

It is impossible due to combinatorial explosion—storing a decision for every percept sequence in Chess requires more memory than there are atoms in the universe. As the agent's lifetime (T) increases, the required lookup table size grows exponentially ,

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

'suck', elif percept['wall\_ahead']: return 'turn\_left', and else: return 'move\_forward'. This logic directly maps immediate sensory inputs to specific behavior without relying on memory or history: detecting food triggers the action to consume it ('suck'), encountering an obstacle directly ahead triggers a directional change ('turn\_left') and

**3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?**

The SimpleReflexAgent got stuck in an infinite loop because it maps current percepts directly to actions without internal state or memory. Due to partial observability, the agent only perceives its immediate adjacent cells, causing distinct grid locations—such

**4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).**

The ModelBasedAgent handles the transition model by tracking executed actions (turn\_left, turn\_right, move\_forward) to update its internal heading, spatial coordinates (x, y), and memory of visited\_cells. It implements the sensor model by using incoming

Finalize and Lock Lab 02 Documentation



**FACULTY OF COMPUTING**